

CSC 254 A4 README

Zhenhao Zhang zzh133@u.rochester.edu

Zhijie Wang zwang179@u.rochester.edu

1. Overview

We use two approaches to parallelize an existing implementation of a simple “N-body” simulation.

1. **NbodyThreads.java**: Thread-based implementation using `CyclicBarrier` for synchronization (Zhenhao responsible)

2. **NbodyExecutor.java**: Executor-based implementation using `ExecutorService` with a fixed thread pool (Zhijie responsible)

Both of us conducted performance testing, analysis, and documentation writing.

Extra Credit:

This feature is not included in the extra credit suggestions.

We added an additional command-line option `-p` to control the number of tasks per thread in executor implementation. So, the total number of tasks becomes:

$$\text{totalTasks} = \text{numThreads} * \text{tasksPerThread}$$

Why increasing total number of tasks helps?

If we only create, for example, 16 tasks for 16 threads, then the execution time might be limited by the slowest task. However, if we increase the number of tasks, each task becomes smaller, and threads that finish early can pick up additional tasks from the executor queue. This reduces idle time and makes the overall performance more stable and efficient.

Through experimentation, validation, and analysis, we determined that the optimal value of **tasksPerThread** is 4. This setting provides the best balance between task granularity and scheduling overhead in our testing environment, resulting in the lowest overall time consume.

Reference:

The executor-based implementation we referenced the `ExecutorService` interface documentation <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ExecutorService.html>

2. Compiling & Running

Compile:

```
javac Nbody.java
javac NbodyThreads.java
javac NbodyExecutor.java
```

Running:

Baseline

```
java Nbody -n 1000 -l 10000 -x 10000
```

Thread-based (-t nThreads)

```
java NbodyThreads 1000 -l 10000 -x 10000 -t 8
```

Executor-based (-p tasksPerThread)

```
java NbodyExecutor -n 1000 -l 10000 -x 10000 -t 8 (by default tasksPerThread = 4)
java NbodyExecutor -n 1000 -l 10000 -x 10000 -t 8 -p 8
```

Note:

Thread number is from 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, and 64.

taskPerThread number is from 2 4 8 16

3. Implementation Details

Thread-Based (NbodyThreads.java)

We create multiple worker threads, and each thread is responsible for calculating forces on a subset of stars. All threads must wait for each other to finish before moving on to the next time step. We use a `CyclicBarrier` to implement this synchronization.

CyclicBarrier:

worker threads run in following steps:

1. `barrier.await()`: Wait until all threads are ready (First wait)
2. Check `shouldStop` flag
3. `doStep()`: Calculation part
4. `barrier.await()`: Wait until all threads have finished their tasks (Second wait)

The two barriers ensure that:

1. No thread starts computing before all threads are ready
2. The main thread doesn't swap the arrays until all computations are complete.

```
// Thread.run() is never invoked directly by user code. Rather, it
// is called by the Java runtime when user code calls Thread.start().
//
public void run() {
    try {
        while (!shouldStop) {
            barrier.await(); // Wait for all threads to start step
            if (shouldStop) break;
            space.doStep(ID, nThreads); // calculate partial step
            barrier.await(); // Wait for all threads to finish step
        }
    } catch (InterruptedException | BrokenBarrierException e) {
        // Exit thread
    }
}
```

Optimization:

To achieve optimal load balancing, stars are partitioned among threads using remainder distribution. The first remaining threads each receive one additional star. This ensures the workload difference between any two threads is at most one star.

```
int starsPerThread = nStars / numThreads;
int remainder = nStars % numThreads;

// remainder stars are distributed one per thread to the first
// threads for load balancing and decrease time waiting at barrier
int startIdx, endIdx;
if (threadId < remainder) {
    startIdx = threadId * (starsPerThread + 1);
    endIdx = startIdx + starsPerThread + 1;
} else {
    startIdx = threadId * starsPerThread + remainder;
    endIdx = startIdx + starsPerThread;
}
```

Executor-Based (NbodyExecutor.java)

The executor-based version assigns thread management to Java's `ExecutorService`. We create a fixed thread pool once, then submit tasks to it each simulation step. The executor handles scheduling and thread reuse automatically.

Executor Initialization:

Complete all allocations in start()

1. newFixedThreadPool(nThreads): Init thread pool
2. Calculate numTasks = nThreads × tasksPerThread
3. ComputeTask: Each task knows which stars it's responsible from startIdx to endIdx
4. (nStars + numTasks - 1) / numTasks: Assign star partitions using ceiling division. So each task gets roughly equal work.

```
// We've been (re)initialized and the user wants us to start
// simulating.
public synchronized void start() {
    running = true;
    killed = false;

    // Initialize thread pool
    executor = Executors.newFixedThreadPool(nThreads);

    // Calculate number of tasks and create task pool
    int numTasks = nThreads * tasksPerThread;
    taskPool = new ComputeTask[numTasks];
    int starsPerTask = (nStars + numTasks - 1) / numTasks;
    // Assign tasks
    for (int i = 0; i < numTasks; i++) {
        int startIdx = i * starsPerTask;
        int endIdx = Math.min(startIdx + starsPerTask, nStars);
        taskPool[i] = new ComputeTask(startIdx, endIdx);
    }
}
```

ComputeTask:

Each ComputeTask is responsible for a range of stars from startIdx to endIdx. Calculate the gravitational force from all other stars. After calculation, write results to otherStars[i]. Because each task only writes within its own index range, while all tasks read from the same input array, the code can safely run in parallel without data races.

```
public void call() {
    for (int i = startIdx; i < endIdx; i++) {
        Star p = stars[i];
        double xForce = 0; double yForce = 0;
        for (int j = 0; j < nStars; j++) {
            if (i != j) { // don't act on self
                double dX = stars[j].xPos - stars[i].xPos;
                double dY = stars[j].yPos - stars[i].yPos;
                double rSquared = dX * dX + dY * dY;
                double rCubed = rSquared * Math.sqrt(rSquared);
                xForce += dX/rCubed;
                yForce += dY/rCubed;
            }
        }
    }
}
```

```

        xForce *= gravity;
        yForce *= gravity;
        Star q = otherStars[i];
        q.xPos = p.xPos + p.xVel + xForce/2.0;
        q.yPos = p.yPos + p.yVel + yForce/2.0;
        q.xVel = p.xVel + xForce;
        q.yVel = p.yVel + yForce;
    }
    return null;
}
}

```

InvokeAll:

Because we do all setups in function start(), we just reuse the same tasks and submit them with invokeAll().

1. invokeAll...(taskPool)): Submit all tasks to the thread pool and wait until every task finishes. Each task runs its call() method in parallel, computing for its assigned stars.
2. Swap stars and otherStars: After all tasks complete, swap the position and run again.

Each task writes to different locations of otherStars[]. No two tasks write to the same location, so there is no need for synchronization during computation.

```

private void doStep() {
    // Submit all tasks and wait for them to finish
    try {
        executor.invokeAll(Arrays.asList(taskPool));
    } catch (InterruptedException e) {
        if (killed) return;
    }
    Star[] tmp = stars;
    stars = otherStars;
    otherStars = tmp;
}

```

4. Results

Thread_based:

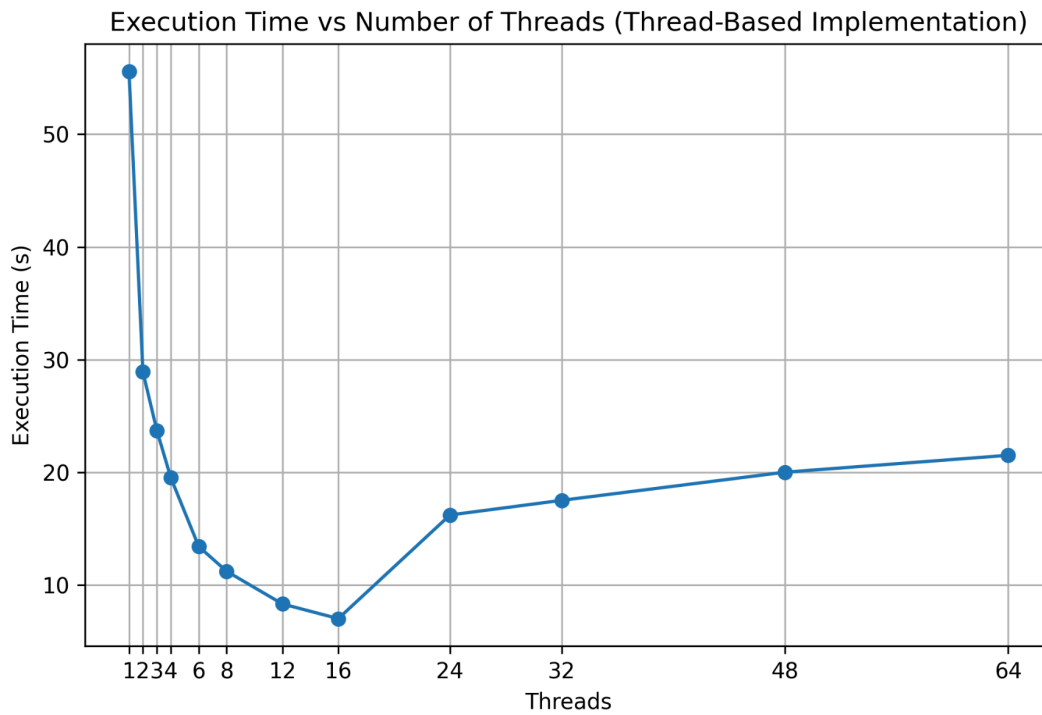


Figure 1: Execution Time vs Number of Threads (Thread-Based Implementation)

Figure 1 shows the execution time of the thread-based implementation as a function of the number of threads.

The execution time decreases rapidly when increasing the thread count from 1 to 8. Between 9 and 16 threads, performance continues to improve, although the ratio is lower. The best performance occurs at 16 threads, where the execution time reaches approximately 7 seconds. Beyond 16 threads, the execution time begins to increase again.

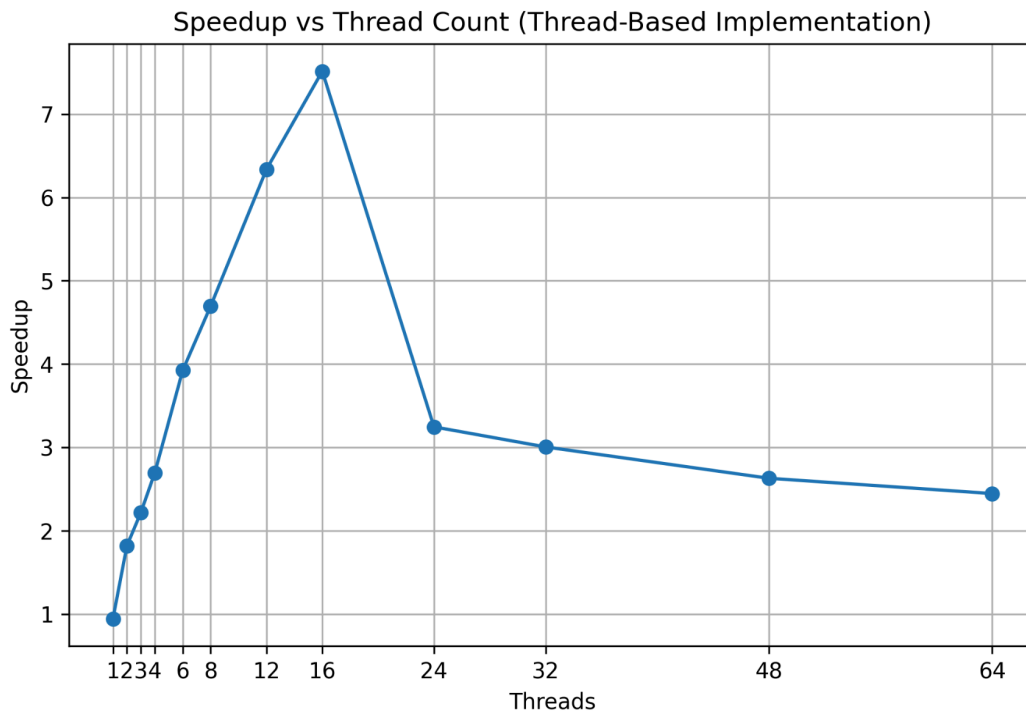


Figure 2: Speedup vs Number of Threads (Thread-Based Implementation)

Figure 2 shows the speedup of the Thread-based implementation relative to the sequential baseline.

Speedup improves quickly from 1 to 8 threads, reaching approximately $4.8\times$ at 8 threads.

Performance continues to scale up to 16 threads, where we observe the maximum speedup of around $7.4\times$, which is the best result across all tested thread counts.

Beyond 16 threads, speedup decreases significantly.

At 24 threads, speedup drops to nearly $3.2\times$, and the number continues to decline as the thread count increases to 32, 48, and 64 threads.

Executor:

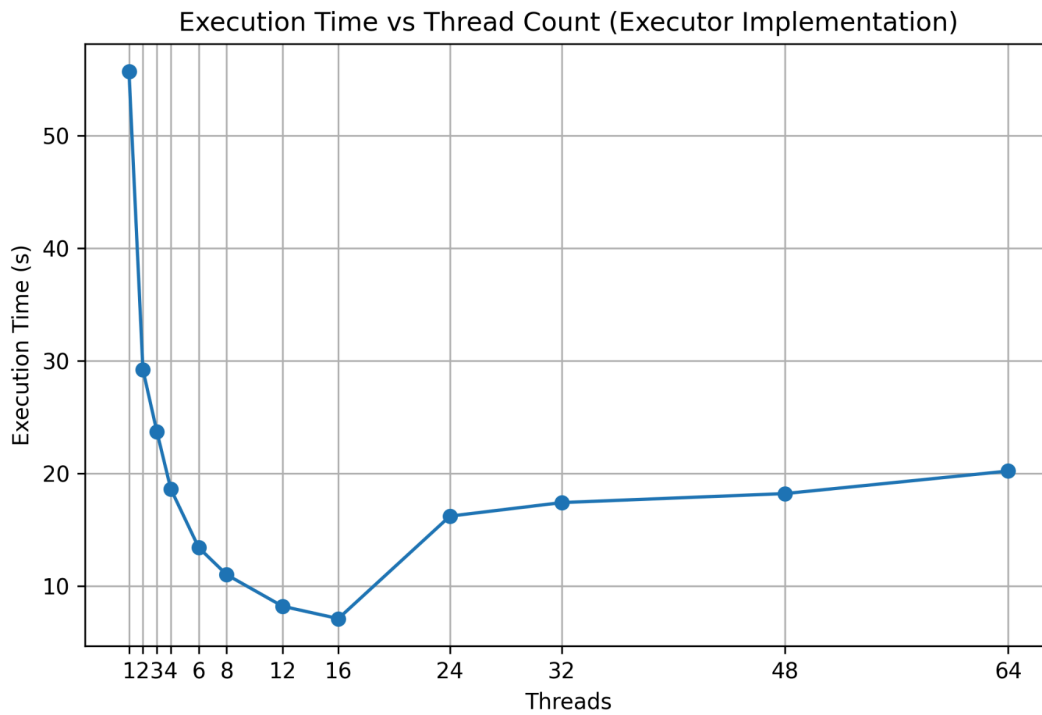


Figure 3: Execution Time vs Number of Threads (Executor Implementation)

Figure 3 shows the execution time of the executor implementation as a function of the number of threads.

The executor implementation shows a similar performance as thread-based implementation: the execution time decreases rapidly when increasing the thread count from 1 to 16 threads. After 16 threads, execution time begins to increase again.

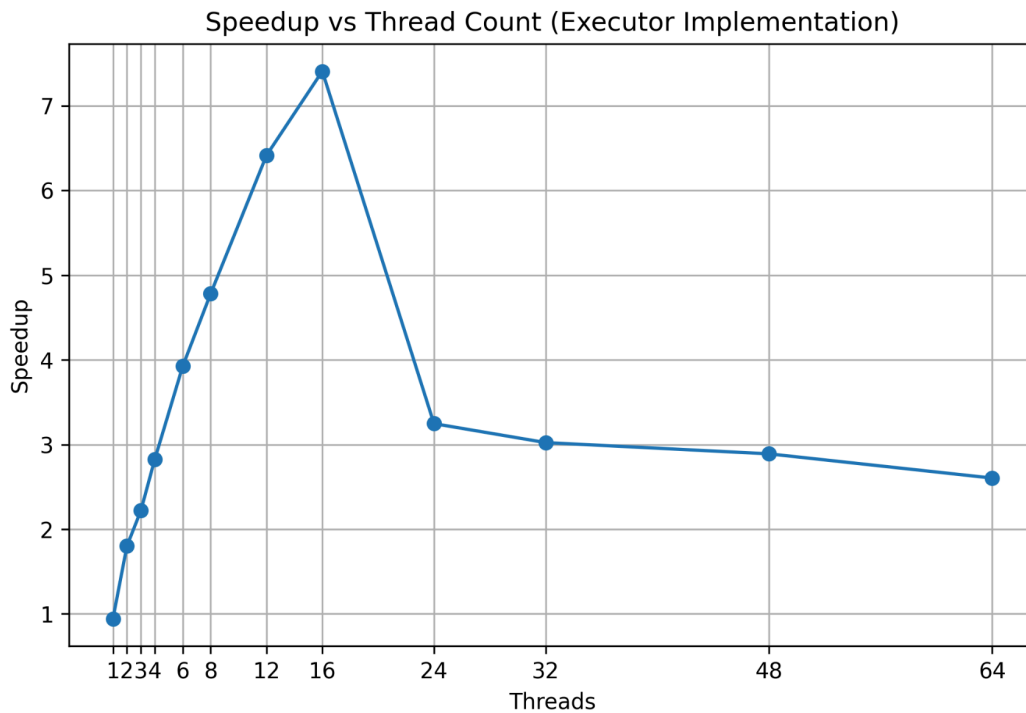


Figure 4: Speedup vs Number of Threads (Executor Implementation)

Figure 4 shows the speedup of the executor implementation relative to the sequential baseline.

Since both executor and thread-based implementations show very similar execution times across all thread counts, their speedup curves are also nearly identical.

The optimal tasksPerThread:

We tested the performance using $N = 2$, $N = 4$, and $N = 8$, with thread counts:

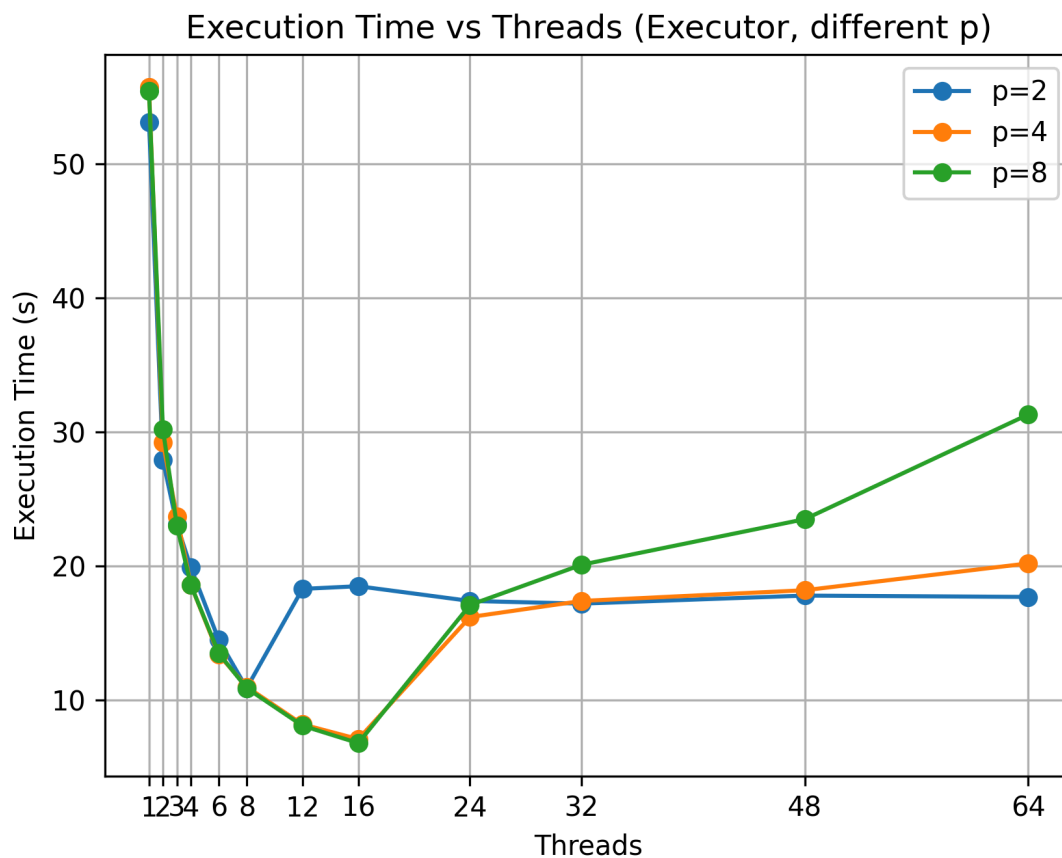
1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64

The execution times (in seconds) are summarized below:

Threads	1	2	3	4	6	8	12	16	24	32	48	64
---------	---	---	---	---	---	---	----	----	----	----	----	----

N = 2	53.1	27.9	23.1	19.9	14.5	10.9	18.3	18.5	17.4	17.2	17.8	17.7
N = 4	55.7	29.2	23.7	18.6	13.4	11.0	8.2	7.1	16.2	17.4	18.2	20.2
N = 8	55.4	30.2	23.0	18.6	13.5	10.9	8.1	6.8	17.1	20.1	23.5	31.3

After computing the total execution time across all configurations, we found that `tasksPerThread = 4` gives the lowest overall runtime. This makes 4 the optimal choice in our environment.



5. Analysis

How close do you come to ideal speedup k?

Our maximum speedup is approximately 7.5x at 16 threads. However, this is still far from the ideal speedup of 16x.

What bottleneck(s) keep you from doing better?

1. Since the increase in thread counts also increases the cache miss, this reduces efficiency.
2. Barrier synchronization is another reason that keeps us doing better. Each simulation step requires all worker threads to stop 2 times. When the thread counts increase, the slowest thread will affect the performance.
3. Another major bottleneck is external interference during testing. Although we followed the recommended testing schedule, some people were still running their tests during my assigned time slot. As a result, the machine was not fully idle, and system load fluctuated significantly.

Do you see any differences between the thread and Executor versions?

The executor implementation performs almost identically to the thread-based version. The execution-time and speedup graphs of both implementations are very close across all thread counts.